

ТЕМА 8. ПОДПРОГРАМИ

УВОД

Разпределение на задачите

- Нека разгледаме един пример – производство на компютри.
- Известно е, че компютър се състои от много компоненти с точно определено предназначение.
- При производството обаче най-често компонентите се произвеждат на друго място, може да са и от друг производител специализиран точно в определената област и се асемблират на етапи.
- Този подход има редица предимства като:
 - Един работник може да се съсредоточи/усъвършенства в една операция без да се разсейва, което ще намали грешките;
 - Едновременно се работи по няколко задачи;
 - Може да се работи независимо, например могат да се поставят компоненти на дънната платка и платките да се складира;
 - Могат да се произвеждат различни модели компютри като се използват различни компоненти от даден тип, например 3 модела, които се различават само по използвания процесор.

УВОД

- Компютърните програми предоставят инструмент на хората да решават задачи по обработка на информация с помощта на КС
- Тази обработка често е много сложна и не се свежда до няколко задачи от рода на сортирай този списък, покажи броя на стоките от даден вид в склада, а се налага да се прави обработка която е много сложна, изисква управление и зависи от множество фактори.
- Вземете за пример един интернет магазин. В такова интернет приложение се реализират стотици възможности за: търсене (по различни критерии), регистрация на потребители, каталогизиране на стоките, следене за наличности, покупки, плащания и мн. др.
- Да вземем например търсене на стока – потребителят може да търси, например по: име, категория, цена, марка, като така критериите за търсене ще бъдат различни, но резултатът от търсенето ще бъде в една форма - списък на стоките, които отговарят на критериите.



- Този пример показва, че програмистът ще има за задача да реализира функционалност за търсене, която се управлява чрез критериите и винаги връща списък със стоки.
- Да вземем друг пример в една банкова система трябва да има функционалност за начисляване на лихва върху депозитите, имаме различни депозити, съответно и лихвените проценти са различни, но формулата за начисляване на лихва е една и съща

$$\text{депозит} = \text{депозит} + \text{депозит} * \text{лихвен процент}$$

- Примерът показва, че не е необходимо програмистът да пише различен код за олихвяване на различните депозити, а кодът е един и същ само трябва да отчита/изчислението се управлява от различния лихвен процент.
- Така сложните задачи могат да се решават с обособен код решаваш по-малка задача.
- Аналогичен е примерът с изчисляването на среден успех за всеки от група студенти(напр. специалност) или пък за дадена дисциплина.

ПОДПРОГРАМИ: АМ – РАЗДЕЛЯЙ И ВЛАДЕЙ

Подпрограмата е първата по-обща абстракция по отношение на обработката на данните въведена в ЕП. Позволява програмиране на модулен принцип.

- **Същност:** Подпрограмите са множество от оператори, които изпълняват определена задача. Те са средство за отделяне и дефиниране на подалгоритъм от даден алгоритъм при програмното му описание.
- **Предназначение:**
 - **групиране на съвкупност от действия**, които са част от даден алгоритъм, обикновено логически самостоятелна част (подалгоритъм). В повечето случаи на тази съвкупност от действия се съпоставя име – **име на подпрограмата**
 - **указване изпълнението** на цялата съвкупност от действия само чрез съпоставеното **име**
 - **Code reuse** – многократно използване на един и същ код.
- С тяхна помощ се позволява **декомпозиране** на проблема на малки и по-лесно решими парчета. Така, чрез всяко малко парче се **решава отделна част** от проблема. Отделните парчета могат да бъдат **изпълнявани в определен ред**.



- Всяка програма обработва данни. Данните обект на обработка са входът на програмата, те се обработват по определен алгоритъм и се получават други данни, които се явяват резултат и които представляват изхода на програмата.
- Нека вземем например програма, която обслужва склад със стоки. В тази програма ще трябва да се реализира функционалност за: въвеждане на данни за артикулите, за справки, търсене на артикул, за сортировка по различни критерии и мн. др.
- Очевидно описаните функционалности са всъщност декомпозирането на проблема и могат да бъдат реализирани като отделни подпрограми, защото обработката на данните в тях не е пряко зависима от обработката в другите (сортировката не зависи от търсенето и т.н.).
- Наред с това, ако търсенето бъде реализирано като сравняване на два низа ще може да се използва за търсене в няколко ситуации - ключовата дума, която е или име на артикул или име на производител или продуктов код или др.

ПОДПРОГРАМИ: ВИДОВЕ

В ЕП се срещат **два вида** подпрограми:

- **Функции** – служат за описание на подалгоритми, чието множество от изходни данни се състои от един елемент, който се нарича *резултат на функцията*
- **Процедури** – използват се за описание на произволни подалгоритми

В C++ има само подпрограми-функции

ПОДПРОГРАМИ: ВЗАИМОДЕЙСТВИЕ МЕЖДУ РАЗЛИЧНИТЕ ЧАСТИ НА ПРОГРАМИТЕ – МЕХАНИЗЪМ НА ПРЕДАВАНЕ НА ПАРАМЕТРИ

- **Видове взаимодействие** за определяне на входните и изходните данни при подпрограмите:
 1. с **външната** среда (потребителя и периферните устройства за вход и изход)
 2. с **вътрешната** среда (с други части на програмата)
- Средство осигуряващо втория вид взаимодействие: "*механизъм на предаване на параметри*"

Механизмът на предаване на параметри осигурява многократното използване на подпрограмите (свойството масовост), давайки възможност подпрограмите да се изпълняват за различни множества от входни данни и да се получават различни изходни резултати

ПОДПРОГРАМИ: МЕХАНИЗЪМ НА ПРЕДАВАНЕ НА ПАРАМЕТРИ

Същността на механизъм на предаване на параметри се състои в това, че задаването на множествата от входни и изходни данни на подпрограмите се разделя на два етапа:

- При описание на съвкупността от действия (деклариране и дефиниране), които подпрограмата групира се използват т.нар. **формални параметри** – служат за определяне на множеството от входни и изходни данни на подпрограмата само в най-общ вид (брой и тип), като им съпоставят формални (условни) имена (без да определят конкретните им стойности)
 - ! Поради специфичното предназначение на функциите, при тях обикновено множеството от изходните данни се състои от един елемент и неговият тип определя типа на резултата на самата функция. Така че при функциите списъкът формални параметри в общия случай определя само множеството от входни данни
- При активирането (изпълнение) на подпрограмата се използват т.нар. **фактически параметри**, които задават конкретните стойности на входните и изходните данни

Възможността формалните параметри на подпрограмите да се заместват с различни фактически параметри при всяко извикване на подпрограмата се нарича механизъм на предаване на параметри

ПОДПРОГРАМИ: ПРЕДИМСТВА

- *Лаконичност* при програмна реализация на алгоритмите, тъй като там където е необходимо да се изпълнява цялата група от действия, е достатъчно да се укаже само името на групиращата подпрограма, без да се налага тази последователност от действия отново да се задава
- *Повишаване надеждността на програмите*, тъй като алгоритъма на задачата се разбива на малки части (подпрограми), които самостоятелно се проектират, програмират, тестват и настройват и по този начин вероятността да се допусне грешка е значително по-малка
- *Многократно използване* на едни и същи подпрограми като елементи при разработване на различни програми. Това от своя страна отново води до повишаване надеждността на програмите, в следствие на възможността за използване на предварително готови и проверени програмни части

ФУНКЦИИ В C++: ЗМ – СИНТАКСИС И СЕМАНТИКА ЗА ДЕФИНИРАНЕ

Синтаксисът се състои от заглавие и блок ({ . . . }):

```
<тип на резултата>   незад  <име> (<списък формални параметри>   незад )  
{  
    <декларация на локални променливи>   незад  
    <оператори>   незад  
    return <израз>;   незад  
}
```

- Ако <тип на резултата> е пропуснат се подразбира `int`. Ако не той може да бъде:
 - всички елементарни типове: `int`, `char`, `float`, и т.н.
 - `void` – функцията не връща резултат
 - указатели (напр. указател към масив)
 - структури
- <име> е името на функцията
- <оператори> описват алгоритъма на функцията
- `return <израз>` служи за връщане на управлението на програмата в точката на извикване на функцията и заедно с това връща нейния резултат, т.е. стойността на <израз> (<израз> е от тип <тип на резултата> или може да се преобразува към него). Тази част може да се пропусне ако ф-ята връща `void`
- ! С този синтаксис, функции не могат да се декларират и дефинират като част от други функции

ФУНКЦИИ В C++: ЗМ – СИНТАКСИС НА СПИСЪКА ФОРМАЛНИ ПАРАМЕТРИ

Синтаксисът допуска 3 основни форми:

- (1) празен списък или `void` – функцията няма входни данни
 - (2) $\langle \text{тип}_1 \rangle \langle \text{име}_1 \rangle = \langle \text{израз}_1 \rangle_{\text{незад}}$, $\langle \text{тип}_2 \rangle \langle \text{име}_2 \rangle = \langle \text{израз}_2 \rangle_{\text{незад}}$ и т.н.
– функцията има определен брой параметри
 - (3) $\langle \text{тип}_1 \rangle \langle \text{име}_1 \rangle = \langle \text{израз}_1 \rangle_{\text{незад}}$, ...
... ,
– т.е. последният параметър е заместен с тогава функцията има променлив брой параметри. Може да се използва ако дефинираме универсална функция, за която и броя и типа на параметрите се променя (напр. като `printf`)
- $\langle \text{тип}_i \rangle \langle \text{име}_i \rangle = \langle \text{израз}_i \rangle_{\text{незад}}$ задава i -ят параметър на ф-ята (съответно – тип, име и евентуално стойност по подразбиране)
 - $\langle \text{тип}_i \rangle$ може да бъде всеки елементарен тип (`int`, `char`, `float`, ...), указател, псевдоним, структура, обединение, клас, масив, функция
 - параметрите със стойност по подразбиране се пишат в края на списъка

ФУНКЦИИ В C++: ЗМ – СЕМАНТИКА НА СПИСЪКА ФОРМАЛНИ ПАРАМЕТРИ

- **Обща семантика:** Задава броя и типа на входните, входно-изходните и изходни данни на функцията и им съпоставя формални имена
- **Семантика според вида на параметъра:**
 - **Формални параметри, които се предават по адрес** – това са параметрите от тип *псевдоним* (&) или *указател* (*). Те служат главно за деклариране на *изходни* или *входно-изходни данни*, въпреки че могат да се използват и за деклариране на входни данни

! За предаване на изходни или входно-изходни параметри по-удобен и по-съвременен е начинът чрез псевдоними
 - **Формални параметри, които се предава по стойност** – това са параметрите, от всички останали типове. Те служат за деклариране само на *входни данни*
 - **Формални параметри-константи** – тези пред типа, на които има служебната дума `const`. Най-често като **параметри-константи** се декларират някои **параметри** от тип *псевдоним* с цел да се използват предимствата на този начин на предаване на параметри (вж. Слайд 10), като в същото време се забрани модификацията им в блока на функцията

ФУНКЦИИ В C++: ЗМ – ПРИМЕРИ ЗА ДЕКЛАРИРАНЕ И ДЕФИНИРАНЕ

- **Параметри по стойност:**

```
float addf(float a, float b)
{ float c=a+b; a = a + 10; return c; }
```

- **Параметри по адрес:**

- (1) чрез *псевдоним* (&):

```
void swap(int& a,int& b){
    int temp=a;
    a=b; b=temp;
}
```

- (1) чрез *указател* (*):

```
void swapP(int* pa,int* pb){
    int temp=*pa;
    *pa=*pb; *pb=temp;
}
```

- **Параметри със стойност по подразбиране:**

```
int init(int count=0, int start=1, int end=100) {
    //...;
    return count;
}
```

- **Параметри-константи:**

```
void PrintInt( const int &IntValue ) {
    printf( "%d\n", IntValue );
}
```

ФУНКЦИИ В C++: ЗМ – ДОПЪЛНИТЕЛНИ ИЗИСКВАНИЯ ЗА БЛОКА НА ФУНКЦИИТЕ

- Тъй като блокът на всяка функция е самостоятелен, то имената на формалните параметри, както и на променливите, константите и т.н. всички програмни елементи, декларирани в блока на функцията е необходимо да бъдат уникални само в рамките на този блок (а не в рамките на цялата програма) Те се наричат *локални*
- При това важи едно ограничение: Ако някое локално име съвпада с глобално, то съответния глобален обект е недостъпен в рамките на тази функция

```
const int c=2;                //глобално c
void fgl(){cout<<endl<< "global: "<<c;};
void main() {
    int c=4;                  //локално c
    cout<<endl<< "local: "<<c;
    fgl();
}
```

- На всички изходни параметри задължително трябва да бъдат присвоени стойности в блока на функцията. Ако функцията връща резултат различен от `void`, то блокът трябва задължително да включва оператор `return`

ФУНКЦИИ В C++: ЗМ – ПРОТОТИПИ НА ФУНКЦИИ

Декларация на функция, която е направена преди и отделно от дефиницията на функцията се нарича **прототипи на функцията (декларация)**

- **Синтаксис** – състои се само от заглавие на функция (вж слайд 8 и 9):

<тип на резултата> <име> (<списък формални параметри> незад) ;

, допуска се имената на формалните параметри да се пропуснат (не е препоръч.)

- **Пример:**

```
int add(int a, int b); //!!! ";" в края на декларацията
int GetIntegerInRange(int low, int high);
```

- **Приложение:**

- Прототипите се използват за инициализация на указатели към функции, преди тези функции да са дефинирани
- Списъкът формални параметри на **прототипа** се използва за проверка на съответствието м/у аргументите от дефиницията и активирането на ф-ята

- Обща практика е всички прототипи на функции да се включват в един заглавен файл (напр. `mylib.h`). Тогава трябва да включим този заглавен файл във всички програмни файлове, които го използват (`#include "mylib.h"`)

- Няма специални изисквания относно реда на деклариране и дефиниране на функциите, дори прототипите могат да се декларират в блока на функцията-клиент
 - Стойностите по подразбиране се задават или само в прототипа или само в дефиницията

ФУНКЦИИ В C++: ЗМ – ОПЕРАТОР ЗА ИЗВИКВАНЕ (АКТИВИРАНЕ) НА ФУНКЦИИ

- **Синтаксис** – той е оператор-израз:

`<име> (<списък фактически параметри> незад)`

- `<име>` е име на предварително декларирана функция
- `<списък фактически параметри>` е последователност от изрази (фактическите параметри, т.е. тези които се подават на ф-ята), разделени с “,” и трябва донякъде да съответстват по реда си, по броя и типа на формалните параметри от описанието
- ако съответните формални параметри имат стойности по подразбиране, които съвпадат с тези на фактическите, то последните могат да се пропуснат, но само ако са в края на списъка

- **Семантика:**

Извикването (`function call`) на ф-ята има типа на резултата на ф-ята и предизвиква нейното изпълнение, като конкретните стойности на входните и изходните данни се определят от зададените при това извикване фактически параметри

ФУНКЦИИ В C++: ЗМ – ПРИМЕРИ ЗА АКТИВИРАНЕ НА ФУНКЦИИ

Активиране на функциите дефинирани на слайд 14:

- **Параметри по стойност:**

```
float x=10, y=20, z=0;
```

```
z = addf(x, y);
```

```
cout<< x <<" , "<< y <<" , "<< z << endl; //изв. 10, 20, 30
```

- **Параметри по адрес:**

- (1) чрез *псевдоним* (&):

```
float x=10, y=20;
```

```
swap(x, y);
```

```
cout<< x <<" , "<< y << endl;
```

```
//ще изведе 20, 10
```

- (1) чрез *указател* (*):

```
float x=10, y=20;
```

```
swapP(&x, &y);
```

```
cout<< x <<" , "<< y << endl;
```

```
//ще изведе 20, 10
```

- **Параметри със стойност по подразбиране:**

```
int i = init(100); //правилно init(100,1,100)
```

```
int j = init(100,,50); //!!!неправилно
```

ФУНКЦИИ В C++: ЗМ – СРАВНЕНИЕ МЕЖДУ ВИДОВЕТЕ ФОРМАЛНИТЕ ПАРАМЕТРИ

Формални параметри, които се предават по адрес (ФПА)

Формални параметри, които се предава по стойност (ФПС)

1. Синтактичните особености:

- а) **ФПА** се декларира като псевдоними или указатели
- б) Фактическите параметри съответни на **ФПС** могат да бъдат произволни изрази, докато тези съответни на **ФПА** – само променливи от съвместим тип

2. Семантични особености:

- а) **ФПА** могат да служат като изходни и като входно-изходни, докато **ФПС** служат само за задаване на входни данни
- б) Измененията, които ф-ята извършва върху **ФПС** не се отразяват върху съответните им фактически параметри, докато при **ФПА** се отразяват

3. Начина на реализацията им в езика C++ (защо така се наричат?):

- а) За фактическите параметри съответни на **ФПС** при извикване на ф-ята се заделя памет в стека. Измененията, които ф-ята извършва с техните стойности се записват в тази памет. След приключване на изпълнението, тази памет се освобождава
- б) За фактическите параметри съответни на **ФПА** в стековата памет се записва само адреса на съответния фактически параметър. Ето защо измененията, които ф-ята извършва с техните стойности се записват в първоначално заделената за тях памет

! Препоръчва се, ако формалните параметри са от някой тип, който заема много памет, те да се декларира като **ФПА** (дори и да са входни), или като параметри-константи

ФУНКЦИИ В C++: ЗМ – ОСОБЕНОСТИ

- В C++ е позволено да се декларират функции с еднакви имена, но с различен списък формални параметри (не се допуска разлика само в типа на резултата), т.е. т.нар. **натоварване на имената на функциите** (*overloading*):

```
float add(float a, float b)
{ return a + b; }
int add(int a, int b)
{ return a + b; }
...
int i,j,k;
float x,y,z;
z = add(x,y); // float add(float a, float b); се извиква
k = add(i,j); // float add(int a, int b); се извиква
```

- В C++ с цел ефективност при изпълнение могат да се декларират т.нар. **inline функции**:

```
inline int add(int a, int b)
{ return a + b; }
```

, което изисква от компилатора извикването на `add(.,.)` да бъде заместено със самия блок на ф-ята, т.е. `a + b` се вмъква (не винаги) в точката на извикването

ФУНКЦИИ В C++: МАСИВИТЕ КАТО ПАРАМЕТРИ НА ФУНКЦИИ

Ако параметър на функция е масив, то:

- се допуска да не се зададе размерът по първото измерение на масива. Компиляторът ще запълни масива според размера на фактическия параметър:

```
int getline(char s[])    {  
    s[i]=c; // or *(s+i)= c;  
    ...  
}
```

В такъв случай е добре като допълнителен параметър да се подаде пропуснатия размер, за да може по-лесно и сигурно да се обработва масива:

```
void print(int a[], int n)  
/* Отпечатва елементите на масив */  
{  
    int i;  
    for (i = 0; i < n; i++)  
        cout << a[i] << " ";  
        cout << "\n";  
}
```

- C++ предава масивите винаги по адрес, независимо че това не е зададено (напр. чрез псевдоним)